Binary Tree Paths

Core Concept

Build complete root-to-leaf paths as strings. DFS traversal, track path, add to result only at leaf nodes.

Key Insights:

- Path = root → ... → leaf (no intermediate nodes)
- Leaf: `node.left == null && node.right == null`
- Build path string as you traverse
- Return immediately after adding leaf path

Approach:

1. Pass current path as string parameter
2. Append current node value to path
3. Check if leaf → add to result & return
4. Append `"->"`
5. Recurse left and right

```java
public List<String> binaryTreePaths(TreeNode root) {
    List<String> result = new ArrayList<>();
    if (root == null) return result;
    dfs(root, "", result);
    return result;
}

private void dfs(TreeNode node, String path,
                 List<String> result) {
    if (node == null) return;

    path += node.val;

    // Leaf node - add complete path
    if (node.left == null && node.right == null) {
        result.add(path);
        return; // Important!
    }

    path += "->";
    dfs(node.left, path, result);
    dfs(node.right, path, result);
}
```

Skeleton Code

Time: O(N × H) - Visit N nodes, string concat O(H)
Space: O(H) - Recursion stack (height)
Output: O(N × H) - Store all paths

Common Mistakes:

- Forgetting `return` after adding leaf
- Adding non-leaf nodes to result
- Not checking for null input

Construct BST from Preorder

Core Concept

Use BST property + range constraints. Each node valid only in `[min, max]`. No inorder needed!

Key Insights:

- BST: left < root < right
- Preorder: root, left subtree, right subtree
- Use range `[min, max]` to validate nodes
- If value out of range → belongs elsewhere
- Global index to track position

Approach:

1. Check if index out of bounds
2. Check if value out of `[min, max]` range
3. Create node, increment index
4. Recurse left: range `[min, val]`
5. Recurse right: range `[val, max]`

```java
class Solution {
    private int index = 0;

    public TreeNode bstFromPreorder(int[] preorder) {
        return build(preorder, Integer.MIN_VALUE,
                     Integer.MAX_VALUE);
    }

    private TreeNode build(int[] preorder,
                           int min, int max) {
        // Base cases
        if (index >= preorder.length) return null;

        int val = preorder[index];
        if (val < min || val > max) return null;

        // Consume element
        index++;
        TreeNode root = new TreeNode(val);

        // Build subtrees with updated ranges
        root.left = build(preorder, min, val);
        root.right = build(preorder, val, max);

        return root;
    }
}
```

Skeleton Code

Time: O(N) - Each element processed once
Space: O(H) - Recursion stack
Why O(N)? Global index ensures single pass

Example Trace:

```
preorder = [8, 5, 1, 7, 10, 12]
build([-ω, +ω]) → 8
  build([-ω, 8]) → 5
    build([-ω, 5]) → 1
    build([5, 8]) → 7
  build([8, +ω]) → 10
    build([10, +ω]) → 12
```

🌲 Build Tree: Preorder + Inorder

Core Concept

Preorder gives root order, inorder splits left/right. Use HashMap for O(1) root lookup.

Key Insights:

- Preorder[0] = root of current subtree
- Find root in inorder → splits left/right
- Left size in inorder = left size in preorder
- Must build LEFT before RIGHT (preorder!)
- HashMap: value → index for O(1) lookup

Approach:

1. Build HashMap: `inorder[i] → i`
2. Pick first from preorder as root
3. Find root position in inorder
4. Elements before root = left subtree
5. Elements after root = right subtree
6. Recurse with inorder boundaries

```java
class Solution {
    private int preIndex = 0;
    private Map<Integer, Integer> inMap =
        new HashMap<>();

    public TreeNode buildTree(int[] preorder,
                              int[] inorder) {
        // Build hashmap for O(1) lookup
        for (int i = 0; i < inorder.length; i++) {
            inMap.put(inorder[i], i);
        }
        return build(preorder, 0, inorder.length - 1)
    ;
    }

    private TreeNode build(int[] preorder,
                           int inLeft, int inRight) {
        if (inLeft > inRight) return null;

        // Pick root from preorder
        int rootVal = preorder[preIndex++];
        TreeNode root = new TreeNode(rootVal);

        // Find split point in inorder
        int inIndex = inMap.get(rootVal);

        // Build left FIRST (preorder sequence!)
        root.left = build(preorder, inLeft,
                          inIndex - 1);
        root.right = build(preorder, inIndex + 1,
                           inRight);

        return root;
    }
}
```

Skeleton Code

Time: O(N) - Visit each node once, O(1) lookup
Space: O(N) - HashMap + O(H) recursion
Without HashMap: $O(N^2)$ - linear search each time

Why Build Left First?

```
Preorder: Root → LEFT → Right
Must consume in this order!
```

Course Schedule (Cycle Detection)

Core Concept

Directed graph cycle detection. Can finish courses no cycles. DFS with "visiting" set.

Key Insights:

- Course dependencies = directed graph
- `[a, b]` means: need b before a
- Cycle = impossible to finish
- `visiting` set = current DFS path
- Revisiting node in path = cycle

Approach:

1. Build adjacency list: course → prerequisites
2. For each course, run DFS
3. If in `visiting` → cycle found
4. If no prerequisites → safe
5. Add to `visiting`, check all prereqs
6. Remove from `visiting` (backtrack)
7. Clear prerequisites (optimization)

```java
class Solution {
    private Map<Integer, List<Integer>> preMap =
        new HashMap<>();
    private Set<Integer> visiting = new HashSet<>();

    public boolean canFinish(int numCourses,
                             int[][] prerequisites) {
        // Build graph
        for (int i = 0; i < numCourses; i++) {
            preMap.put(i, new ArrayList<>());
        }
        for (int[] prereq : prerequisites) {
            preMap.get(prereq[0]).add(prereq[1]);
        }

        // Check each course
        for (int c = 0; c < numCourses; c++) {
            if (!dfs(c)) return false;
        }
        return true;
    }

    private boolean dfs(int crs) {
        // Cycle detected!
        if (visiting.contains(crs)) return false;

        // No prerequisites
        if (preMap.get(crs).isEmpty()) return true;

        visiting.add(crs);
        for (int pre : preMap.get(crs)) {
            if (!dfs(pre)) return false;
        }
        visiting.remove(crs); // Backtrack
        preMap.put(crs, new ArrayList<>()); //
     Optimize

        return true;
    }
}
```

Skeleton Code

Time: O(V + E) - V courses, E prerequisites
Space: O(V + E) - Graph + O(V) recursion
Why not O(V²)? Clear prereqs after first visit

Three States:

- Unvisited (white) - not checked
- Visiting (gray) - in current path
- Visited (black) - fully explored (empty prereqs)

Reverse Linked List (Recursive)

Core Concept

Recursion to deepest node, return as new head. Reverse pointers while unwinding.

Key Insights:

- Base case: single node or null
- Recurse to end, that becomes new head
- Same `newHead` bubbles up all calls
- Reverse pointer: `head.next.next = head`
- Break old pointer: `head.next = null`

Approach:

1. Base: if null or single node, return it
2. Recurse on `head.next`
3. Receive new head from recursion
4. Reverse: make next point back to current
5. Break forward pointer
6. Return same new head up

```java
public ListNode reverseList(ListNode head) {
    // Base case
    if (head == null || head.next == null) {
        return head;
    }

    // Recurse to end
    ListNode newHead = reverseList(head.next);

    // Reverse pointer
    head.next.next = head;

    // Break old pointer
    head.next = null;

    // Return new head
    return newHead;
}
```

Skeleton Code

Time: O(N) - Visit each node once
Space: O(N) - Recursion stack
Iterative: O(1) space possible

Execution Flow:

`Input: 1→2→3→4→null`

```
reverseList(1) calls reverseList(2)
reverseList(2) calls reverseList(3)
reverseList(3) calls reverseList(4)
reverseList(4) returns 4 (base case)
↓
reverseList(3): 3.next.next = 3
                3.next = null
                return 4
↓
reverseList(2): 2.next.next = 2
                2.next = null
                return 4
↓
reverseList(1): 1.next.next = 1
                1.next = null
                return 4

Result: null←1←2←3←4
```

Common Mistakes:

- Forgetting to break old pointer
- Not returning same `newHead`
- Reversing in wrong order

Problem-Solving Patterns

Tree Problems

When to use DFS:

- Need complete paths (root to leaf)
- Tree traversal (pre/in/post order)
- Searching for specific nodes

When to use BFS:

- Level-order traversal
- Shortest path in tree
- Level-by-level processing

Common Tree Patterns:

```java
// Pattern 1: Leaf check
if (node.left == null && node.right == null) {
    // Process leaf
}

// Pattern 2: Null check first
if (node == null) return;

// Pattern 3: Base case before recursion
if (baseCondition) return result;
recursive_call();

// Pattern 4: Global variable vs return value
// Use global for: paths, counts, flags
// Use return for: node references, subtree info
```

Graph Problems

Cycle Detection (Directed):

- Use `visiting` set (current path)
- If revisit → cycle
- Remove from `visiting` after exploring

Topological Sort:

- DFS: Add to result in post-order
- Kahn's: BFS with in-degrees
- Only exists if no cycles

Graph Representations:

```java
// Adjacency List (most common)
Map<Integer, List<Integer>> graph;

// Adjacency Matrix (dense graphs)
int[][] graph = new int[n][n];

// Edge List (for some algorithms)
List<int[]> edges;
```

Recursion Patterns

Key Concepts:

- Base case(s) first
- Recursive case with smaller problem
- Trust the recursion!
- Think: "What if subproblem solved?"

Return Flow:

- Returns to **exact line** that called it
- Execution continues from that point
- Not to the function itself!

State Management:

- Pass state down: parameters
- Collect state up: return values
- Track state: global variables
- Backtrack: undo changes after recursion

Complexity Quick Reference

Common Time Complexities

Operation	Complexity
Tree traversal (DFS/BFS)	O(N)
Binary search	O(log N)
Linear search	O(N)
Sorting	O(N log N)
HashMap get/put	O(1) avg
String concatenation	O(length)
Array copy	O(N)

Space Complexity Factors

- **Recursion stack:** O(H) for trees, O(N) worst
- **HashMap/Set:** O(N) for N elements
- **Result storage:** Usually not counted
- **Input space:** Not counted

Tree Height

- Balanced tree: H = O(log N)
- Skewed tree: H = O(N)
- Complete tree: H = O(log N)

Debugging Checklist

Before Coding:
- Understand problem completely
- Draw examples (especially edge cases)
- Identify pattern/algorithm
- Consider time/space constraints

While Coding:
- Handle null/empty inputs
- Check array bounds
- Initialize data structures
- Consider base cases

Common Bugs:
- Off-by-one errors (index ± 1)
- Not handling null
- Wrong loop bounds
- Forgetting to return
- Not clearing/resetting state
- Wrong recursion order

Testing Strategy:
1. Empty input
2. Single element
3. Two elements
4. Normal case
5. Edge case (max/min values)

Interview Communication

Problem-Solving Process:
1. **Clarify:** Ask questions, confirm understanding
2. **Example:** Work through small example
3. **Approach:** Explain high-level strategy
4. **Complexity:** Analyze before coding
5. **Code:** Write clean, readable code
6. **Test:** Walk through with examples
7. **Optimize:** Discuss improvements

Communication Tips:
- Think out loud
- Explain your reasoning
- Acknowledge trade-offs
- Ask for hints if stuck
- Stay calm, be methodical

Code Templates

DFS Tree Traversal

```
void dfs(TreeNode node, /* params */) {
    if (node == null) return;

    // Pre-order: process here

    dfs(node.left, /* params */);

    // In-order: process here

    dfs(node.right, /* params */);

    // Post-order: process here
}
```

Graph DFS (Cycle Detection)

```
boolean dfs(int node) {
    if (visiting.contains(node)) return false;
    if (visited.contains(node)) return true;

    visiting.add(node);
    for (int neighbor : graph.get(node)) {
        if (!dfs(neighbor)) return false;
    }
    visiting.remove(node);
    visited.add(node);

    return true;
}
```

BFS Template

```
void bfs(TreeNode root) {
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            // Process node

            if (node.left != null)
                queue.offer(node.left);
            if (node.right != null)
                queue.offer(node.right);
        }
        // Level complete
    }
}
```

Binary Search Template

```java
int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }

    return -1;
}
```

Backtracking Template

```java
void backtrack(/* state */) {
    if (/* goal */) {
        // Add to result
        return;
    }

    for (/* choices */) {
        // Make choice
        backtrack(/* new state */);
        // Undo choice
    }
}
```

Algorithm Comparison

Tree Construction

Given	Time	Space
Preorder + Inorder	O(N)	O(N)
Preorder (BST only)	O(N)	O(H)
Level-order + Inorder	O(N)	O(N)
Preorder only	Impossible	-

Graph Traversal

Algorithm	Time	Space	Use Case
DFS	O(V+E)	O(H)	Paths, Cycles
BFS	O(V+E)	O(W)	Shortest path
Topological	O(V+E)	O(V)	Ordering

Cycle Detection

Graph Type	Method	Key
Directed	DFS visiting set	Gray node
Undirected	DFS parent track	Back edge
Linked List	Floyd's	Fast/slow

Edge Cases Checklist

Tree Problems

- Null root
- Single node
- Only left children (skewed)
- Only right children (skewed)
- Balanced vs unbalanced
- Duplicate values (if allowed)

Graph Problems

- No edges (disconnected nodes)
- Self-loops
- Multiple edges between nodes
- Disconnected components
- Single node
- Cycles

Array Problems

- Empty array
- Single element
- All same elements
- Sorted vs unsorted
- Negative numbers
- Integer overflow

String Problems

- Empty string
- Single character
- Special characters
- Unicode characters
- Case sensitivity

Optimization Techniques

Time Optimization

1. Use HashMap for O(1) Lookup

```
// Instead of linear search
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < arr.length; i++) {
    map.put(arr[i], i);
}
int index = map.get(target); // O(1)
```

2. Early Termination

```
// Stop as soon as answer found
for (...) {
    if (condition) return result;
}
```

3. Memoization

```
Map<Key, Value> memo = new HashMap<>();
if (memo.containsKey(key)) {
    return memo.get(key);
}
// Compute result
memo.put(key, result);
return result;
```

4. Two Pointers

```
int left = 0, right = arr.length - 1;
while (left < right) {
    // Process and move pointers
}
```

Space Optimization

1. Reuse Input

```
// Modify array in-place instead of creating new
```

2. Use Indices Instead of Copies

```
// Pass (start, end) instead of array slices
helper(arr, start, end);
```

3. Iterative vs Recursive

```
// Iterative uses O(1) space vs O(N) recursion
Stack<Node> stack = new Stack<>();
// ... iterative traversal
```

4. Bit Manipulation

```
// Use bits for boolean flags
int flags = 0;
flags |= (1 << i);  // Set bit i
if ((flags & (1 << i)) != 0) // Check bit i
```

✖ Common Mistakes & Fixes

Index Errors

```java
// Off-by-one
for (int i = 0; i < arr.length; i++) {
    if (arr[i+1] > arr[i]) // ArrayIndexOutOfBounds!
}
```

Wrong

```java
for (int i = 0; i < arr.length - 1; i++) {
    if (arr[i+1] > arr[i]) // OK
}
```

Correct

Null Pointer

```java
void process(TreeNode node) {
    int val = node.val; // NPE if node is null!
}
```

Wrong

```java
void process(TreeNode node) {
    if (node == null) return;
    int val = node.val; // Safe
}
```

Correct

Infinite Recursion

```java
int fibonacci(int n) {
    return fibonacci(n-1) + fibonacci(n-2);
    // No base case!
}
```

Wrong

```java
int fibonacci(int n) {
    if (n <= 1) return n; // Base case
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Correct

Modifying While Iterating

```java
for (Integer num : list) {
    if (num < 0) list.remove(num);
    // ConcurrentModificationException!
}
```

Wrong

```java
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    if (it.next() < 0) it.remove();
}
```

Correct

Integer Overflow

```java
int mid = (left + right) / 2;
// Overflow if left + right > MAX_INT
```

Wrong

```java
int mid = left + (right - left) / 2;
// Safe from overflow
```

Correct

Essential Data Structures

Java Collections Cheatsheet

Structure	Implementation	Use Case
List	ArrayList LinkedList	Random access Insert/delete
Set	HashSet TreeSet	O(1) lookup Sorted order
Map	HashMap TreeMap	Key-value O(1) Sorted keys
Queue	LinkedList PriorityQueue	FIFO Min/max heap
Stack	Stack/Deque	LIFO

Time Complexities

Operation	ArrayList	LinkedList	HashMap
get(i)	O(1)	O(N)	O(1)
add(end)	O(1)*	O(1)	O(1)
add(i)	O(N)	O(N)	-
remove(i)	O(N)	O(N)	O(1)
contains	O(N)	O(N)	O(1)

* *Amortized*

Choosing Data Structure

Need fast lookup by key? → HashMap
Need ordering? → TreeSet/TreeMap
Need duplicates + order? → List
Need unique elements? → Set
Need FIFO? → Queue
Need LIFO? → Stack
Need min/max quickly? → PriorityQueue

Useful Formulas

Tree Properties

- Max nodes at level L: 2^L
- Max nodes in tree height H: $2^{H+1} - 1$
- Min height for N nodes: $\lceil \log_2(N+1) \rceil - 1$
- Leaf nodes in full binary tree: $(N+1)/2$

Graph Properties

- Max edges (directed): $V \times (V-1)$
- Max edges (undirected): $V \times (V-1)/2$
- Tree edges: $V-1$
- Complete graph edges: $V2 = V(V-1)/2$

Complexity Bounds

- $O(1) < O(\log N) < O(N) < O(N \log N)$
- $O(N \log N) < O(N^2) < O(2^N) < O(N!)$
- $\log_2(1000000) \approx 20$
- $\sqrt{1000000} = 1000$

Practice makes perfect! Good luck! ☻
Remember: Understand the pattern, not just memorize the code.